

DevOps-, DataOps- und CI/CD-Grundlagen

Section 5 · Implementing CI-CD · Quelle: Kurs 4, M02 Kapitel 5–7

Bevor man Continuous-Integration- und Continuous-Deployment-Praktiken (CI/CD) konkret in Databricks umsetzt, lohnt sich ein Blick auf die dahinterliegenden Konzepte: DevOps, sein datenspezifisches Pendant DataOps, und die Rolle von CI vs. CD innerhalb dieses Lebenszyklus.

1. Was ist DevOps?

DevOps ist eine Kultur und eine Sammlung von Praktiken, die Software-Engineering-Best-Practices mit IT-Betrieb (Operations) verbindet, um Software schneller, zuverlässiger und in höherer Qualität auszuliefern. Der klassische DevOps-Lebenszyklus umfasst acht wiederkehrende Phasen: *Planen* → *Entwickeln (Code)* → *Bauen (Build)* → *Testen* → *Freigeben (Release)* → *Deployen* → *Betreiben* → *Überwachen (Monitor)* – und beginnt danach wieder von vorn.

2. DataOps = DevOps für die Data-Engineering-Welt

DataOps überträgt genau diese Prinzipien – Automatisierung, Zusammenarbeit, kontinuierliche Verbesserung – auf Datenpipelines: Es automatisiert das Management von Datenpipelines, überwacht deren Qualität und beschleunigt die Auslieferung neuer/geänderter Pipelines in die Produktion. Parallel dazu überträgt **MLOps** dieselben Prinzipien auf den Machine-Learning-Lebenszyklus (Modelltraining, -versionierung, -deployment). Alle drei Disziplinen teilen sich dieselbe Grundidee, unterscheiden sich aber in den konkreten Artefakten, die durch die Pipeline fließen: Code (DevOps), Daten (DataOps) bzw. Modelle (MLOps).

3. Die Rolle von CI und CD

Continuous Integration (CI) bedeutet, dass Codeänderungen mehrerer Mitwirkender regelmäßig in ein zentrales Repository zusammengeführt (gemerged) und dabei automatisiert getestet werden – Tests, die fehlschlagen, verhindern, dass fehlerhafter Code in den Hauptzweig gelangt. Dabei orientiert man sich häufig an der sogenannten *Testpyramide*: an der Basis stehen viele schnelle **Unit-Tests** (einzelne Funktionen isoliert), darüber weniger, aber breitere **Integrationstests** (Zusammenspiel mehrerer Komponenten, z. B. einer ganzen Pipeline), an der Spitze wenige **Systemtests** (das Gesamtsystem end-to-end).

Continuous Delivery automatisiert das Ausrollen geprüfter Änderungen in eine Staging-/Vorproduktionsumgebung, überlässt den finalen Schritt nach Produktion aber einer manuellen Freigabe. **Continuous Deployment** geht einen Schritt weiter: Jede Änderung, die alle Tests besteht, wird automatisch bis in die Produktionsumgebung ausgerollt – ganz ohne manuellen Eingriff. Welche der beiden Varianten sinnvoll ist, hängt vom Risiko- und Reifegrad des jeweiligen Projekts ab.

4. Umgebungsisolation als Grundlage

Damit CI/CD funktioniert, müssen Entwicklungs-, Test- (Stage) und Produktionsumgebung sauber voneinander getrennt sein – in Databricks typischerweise über separate Unity-Catalog-Kataloge (`dev` / `stage` / `prod`) mit jeweils eigenen Berechtigungen, sodass Entwickler nie versehentlich Produktionsdaten verändern.

Ergänzt aus der offiziellen Databricks-Dokumentation: Für das Testen von Lakeflow Declarative Pipelines über mehrere Umgebungen hinweg empfiehlt Databricks, gezielt Entwicklungs- und Testdatensätze mit sowohl validen als auch bewusst fehlerhaften Datensätzen anzulegen, um die Pipeline-Logik (inkl. Expectations) gegen realistische Fehlerfälle zu prüfen; jeder Pipeline-Lauf schreibt zudem ein strukturiertes Event-Log (als Delta-Tabelle abfragbar), das sich für automatisierte Prüfungen über `dev`/`stage`/`prod` hinweg auswerten lässt.

Quelle: [Manage data quality with pipeline expectations – Databricks-Dokumentation](#)